Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

$\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$

Formal Specification and Documentation using Z:

A Case Study Approach

A Brief Introduction to Z

Jonathan Bowen

Department of Computer Science
University of Reading

Email: *J.P.Bowen@reading.ac.uk*

URL:

http://www.comlab.ox.ac.uk/oucl/users/jonathan.bowen/zbook.html

$\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$ $\mathbb{Z}$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

A Brief Introduction to Z

- Typed formal specification notation
- First order predicate logic
- Zermelo-Fraenkel (ZF) set theory
- Originated by J-R Abrial;
  developed at the Oxford University Computing Laboratory
- Popular with governments, academics and some industry
- International standardization under ISO/IEC JTC1/SC22
- Z User Group organizes regular meetings

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Predicate Logic

Predicates are *true* or *false* – no 'undefined' value ≀≀≀

Propositional Logic

Connectives:

Unary logical negation: '$\neg\, p$'

Infix binary connectives:

- Logical conjunction: '$p \land q$'
- Logical disjunction: '$p \lor q$'
- Logical implication: '$p \Rightarrow q$'
  (same as '$\neg\, p \lor q$')
- Logical equivalence: '$p \Leftrightarrow q$'

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Quantification

Over a list of variables X

- Universal quantification: $\forall X \bullet q$
- Existential quantification: $\exists X \bullet q$
- Unique existential quantification: $\exists_1 X \bullet q$

$\wr\wr\wr$ Scope of the variables listed in X is bounded by the clause q.

Laws

Rich set of algebraic laws for transforming predicates for proofs.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Sets and Relations

Types

1. Help to structure specifications. E.g.:

   $$x : Path$$

2. Refinement into code.

3. Helps in avoiding nonsense specifications.

   Consistency checking.
   E.g., given $Z : Methods$ and
   $Jonathan : People$,
   $Z = Jonathan$ is meaningless.

Consider the *schema* box:

$$\begin{array}{|l} A, B : People \\ x : Likes \\ \hline \mathsf{Predicate}(x) \\ \hline \end{array}$$

First part: *signature*
Second part: *predicate*

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Sets

Collection of objects or *elements* of some type.

Examples:

$\emptyset$ The *empty set* – also denoted as $\{\}$

$\{Jane, Alice, Emma\}$ Three (type-compatible!) elements

$\{0,1,2\} = \{0,0,1,0,2,1\} = \{1,2,0\}$ Same three element set of numbers (no ordering)

$\{0,1,2,3,\ldots\}$ Set of *natural numbers* – infinite number of elements, denoted $\mathbb{N}$ for brevity

$\{\emptyset\} \neq \emptyset$ Set containing the empty set is *not* empty

Set elements drawn from some *basic type* (or *given set*) in Z (even the empty set).

$\emptyset[T]$ indicates empty set drawn from type T.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Set membership

x is a member of S: $x \in S$

Examples:

$0 \in \{0, 1, 2\}$

$\emptyset \in \{\emptyset\}$

$0 \notin \emptyset$ Alternative to '$\neg\ (0 \in \emptyset)$'

In fact for any x: $x \notin \emptyset$

$William \notin \{Jonathan, Jane, Alice, Emma\}$

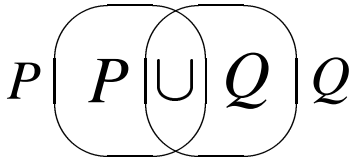
Question: Is $\{0\} \in \{\{0, 1, 2\}\}$ true?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Operations on sets

$P \cup Q$ '*P union Q*' – union.

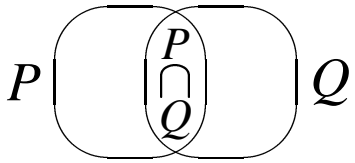
All elements in either P or Q (or both).



$\{a, b, c, \ldots\}$ is shorthand for $\{a\} \cup \{b\} \cup \{c\} \cup \ldots$

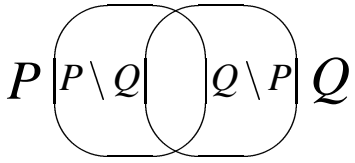
$P \cap Q$ '*P intersect Q*' – intersection.

Elements in both P and Q.



$P \setminus Q$ '*P minus Q*' – difference.

Elements in P, but in Q.



Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Predicates on sets:

$P = Q$ 'P *equals* Q' – identity of sets.

P and Q have the same elements.

≀≀≀ No repeated elements and no order:

$$\{Alice\} = \{Alice, Alice, Alice\}$$
$$\{Alice, Emma\} = \{Emma, Alice\}$$

$$(P \setminus Q) \cup (P \cap Q) \cup (Q \setminus P) = P \cup Q$$

Negation of $P = Q$

P and Q are *not* the same.

'$P \neq Q$' (equivalent to '$\neg\ (P = Q)$').

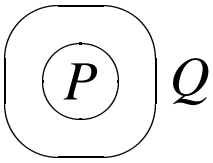
Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

$P \subseteq Q$ '*P contained in Q*' – subset.

Elements of P are in Q. (Perhaps $P = Q$.)

Strict subset: '$P \subset Q$'

Shorthand for '$P \subseteq Q \land P \neq Q$'.

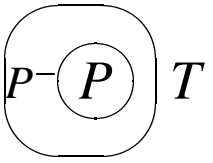


P^- '*complement P*' – complementation.

All the elements *not* in a set.

For a type T where $P \subseteq T$:

$$P^- = T \setminus P$$



$T^- = \emptyset[T]$ and $\emptyset[T]^- = T$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Set membership

$x \in P$ '*x belongs to P*' – membership.

Also '$x \notin P$' (i.e., '$\neg\,(x \in P)$').

Question: Simplify:

1. $(P \setminus Q) \setminus P$
2. $(P \setminus Q) \cup (P \cap Q)$

Generalized set operations

Set union and intersection on a set of sets:

$$\bigcup\{A, B, C, \ldots\} = A \cup B \cup C \cup \ldots$$
$$\bigcap\{A, B, C, \ldots\} = A \cap B \cap C \cap \ldots$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Set comprehension

General way of specifying sets:

$$\{x : \text{Type} \mid \text{Predicate}(x)\}$$

or more generally $\{\text{Signature} \mid \text{Predicate}\}$

Examples:

$\{x : \mathbb{N} \mid x\, is_prime\}$ (= $\{2, 3, 5, 7, 11, 13, \ldots\}$).

$\{x : Path \mid least_cost(x)\}$

$\{x : Methods \mid Jonathan\ \underline{uses}\ x\}$

Question: How could we write a definition for $\emptyset$ using set comprehension?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Subsets

$A \subseteq B$

$\emptyset \subseteq A$ Always true.

$A \subseteq A$ Every set 'contains' itself.

$\{0, 1\} \subseteq \mathbb{N}$

$\{x : \mathbb{N} \mid x \mathit{is_prime} \land x \neq 2\} \subseteq \{x : \mathbb{N} \mid x \mathit{is_odd}\}$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Sets and predicates

Set intersection ($\cap$) and logical conjunction ($\wedge$):

$$\{x : T \mid p\} \cap \{x : T \mid q\} = \{x : T \mid p \wedge q\}$$

Union ($\cup$) and disjunction ($\vee$):

$$\{x : T \mid p\} \cup \{x : T \mid q\} = \{x : T \mid p \vee q\}$$

Complementation and negation:

$$\{x : T \mid p\}^{-} = \{x : T \mid \neg\, p\}$$

≀≀≀ N.B., T must be a basic type.

Subset ($\subseteq$) and implication ($\Rightarrow$):

$$\{x : T \mid p\} \subseteq \{x : T \mid q\} \text{ iff } p \Rightarrow q$$

Set equality ($=$) and logical equivalence ($\Leftrightarrow$):

$$\{x : T \mid p\} = \{x : T \mid q\} \text{ iff } p \Leftrightarrow q$$

Empty set (of a particular type T) and *false* predicate:

$$\emptyset[T] = \{x : T \mid false\}$$

Given type and *true* predicate:

$$T = \{x : T \mid true\}$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Set operations

Subset operator $\subseteq$ – mathematical properties. For sets P, Q and R, of type T:

$\subseteq$ is *reflexive*: $P \subseteq P$
$\subseteq$ is *transitive*: $(P \subseteq Q \land Q \subseteq R) \Rightarrow P \subseteq R$
$\subseteq$ is *antisymmetric*: $(P \subseteq Q \land Q \subseteq P) \Rightarrow P = Q$
$\emptyset[T]$ is the *minimum* of T: $\emptyset[T] \subseteq P$

Set intersection ($\cap$) properties:

- $\cap$ is the *greatest lower bound* of $\subseteq$:

  $$R \subseteq P \land R \subseteq Q \Leftrightarrow R \subseteq (P \cap Q)$$

  $P \cap Q$ is the largest subset of P and Q.

- $\cap$ is *idempotent* $P \cap P = P$
  symmetric $P \cap Q = Q \cap P$
  associative $(P \cap Q) \cap R = P \cap (Q \cap R)$
  monotonic $P \subseteq Q \Rightarrow (R \cap P) \subseteq (R \cap Q)$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Complement of P (with respect to its type).

Involutive:

$$(P^-)^- = P$$

Cf. *contrapositive* in predicate logic:

$$P \subseteq Q \Leftrightarrow Q^- \subseteq P^-$$

Note that

$$P \subseteq Q \Leftrightarrow P \cap Q^- = \emptyset$$

$P \cup Q$ is $(P^- \cap Q^-)^-$
(one of De Morgan's laws)

Further properties for $\cup$, etc.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Cartesian product

$$T \times U$$

denotes the type of ordered pairs (t, u) with $t : T$ and $u : U$.
If $P \subseteq T$ and $Q \subseteq U$ then

$$P \times Q = \{p : T; \; q : U \mid p \in P \land q \in Q\}$$

Generalizable to an ordered *n-tuple*:

$$E_1 \times E_2 \times \ldots \times E_n$$

Examples:

$$(Z, Jonathan) \in Methods \times People$$

$$(Oxford, Cambridge) \in Places \times Places$$

$$(Jonathan, Oxford, 1956) \in People \times Places \times \mathbb{N}$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

first and or *second* element from an ordered pair:

$$first(t, u) = t$$
$$second(t, u) = u$$

For an ordered pair t: $(first\ t, second\ t) = t$

Question: Is $X \times Y \times Z$ equivalent to $(X \times Y) \times Z$ or $X \times (Y \times Z)$?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Set comprehension revisited

$$\{ \text{Signature} \mid \text{Predicate} \}$$

For example:

$$\{x : Methods;\ y : People \mid Jonathan\ uses\ x \land x\ approved_by\ y\}$$

Signature – *ordered* tuple of type
$Methods \times People$

≀≀≀ The order is important:

$$x : Methods;\ y : People$$

is not the same as

$$y : People;\ x : Methods$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Extraction of parts of the signature:

$$\{x_1 : T_1; \ \ldots; \ x_n : T_n \mid \text{Predicate}\}$$

is the same as

$$\{x_1 : T_1; \ \ldots; \ x_n : T_n \mid \text{Predicate} \bullet (x_1, \ldots, x_n)\}$$

Set of squares of primes:

$$\{x : \mathbb{N} \mid x \mathit{is_prime} \bullet x * x\}$$

In general:

$$\{\ \text{Signature} \mid \text{Predicate} \bullet \text{Expression}\ \}$$

Signature – variable declarations
Predicate – constraints
Expression – the set elements

Expression omitted if a tuple formed from the signature.
Predicate omitted if *true*.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Power set

For a set S, $\mathbb{P}S$ denotes the set of all subsets (*power set*) of S.

$$X \in \mathbb{P}S \Leftrightarrow X \subseteq S$$

$\emptyset \in \mathbb{P}S$, so $\mathbb{P}S \neq \emptyset$.

Examples:

$\mathbb{P}\emptyset = \{\emptyset\}$

$\mathbb{P}\{a\} = \{\emptyset, \{a\}\}$

$\mathbb{P}\mathbb{N}$ Sets of natural numbers

$\mathbb{P}(\mathbb{N} \times \mathbb{N})$ Sets of pairs of numbers

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

A power set can include infinite subsets. If S is a set, $\mathbb{F}\, S$

$S == \mathbb{P}((Places \times Places) \times Path)$

Abbreviation definition

$X == Y$ means 'replace X by Y' subsequently.

Brackets used for grouping and nesting – use sparingly.

$_ \rightarrowtail _ \in S$ – underscores are place-holders for parameters; $\rightarrowtail$ infix operator.

A power set can include infinite subsets.
If S is a set, $\mathbb{F}\, S$ denotes the set of all *finite* subsets of S.

Non-empty subsets: $\mathbb{P}_1$ (non-empty power set)
or $\mathbb{F}_1$ (non-empty set of finite subsets)

$\mathbb{P}_1 S == \mathbb{P} S \setminus \{\emptyset\}$

$\mathbb{F}_1 S == \mathbb{F} S \setminus \{\emptyset\}$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Relations

Examples of relationships (two-place predicates):

R_1: Places 'A' and 'B' are adjacent.

R_2: Path x costs y to traverse.

R_3: $(A, B) \rightarrowtail P$ (P is a path from A to B of least cost.)

In general: $R \subseteq P \times Q$

More formally:

$(A, B) \in R_1$ iff A,B adjacent.

$(x, y) \in R_2$ iff path x costs y.

$((A, B), P) \in R_3$ iff P of least cost A to B.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Infix relations:

$$p\ R\ q \quad \text{instead of} \quad (p, q) \in R$$

(For example, '$s \rightarrowtail t$'.) $_R_$ indicates an infix relation.

Maplet notation:

$$p \mapsto q$$

instead of (p, q).

Example relations:

$$_ \leq _ = \{x, y : \mathbb{N} \mid (\exists z : \mathbb{N} \bullet x + z = y)\}$$

$$\begin{aligned} uses \ &= \{Jonathan \mapsto Z, Peter \mapsto VDM, \\ &\quad Peter \mapsto RAISE\} \end{aligned}$$

If T, U are types,

$$T \leftrightarrow U == \mathbb{P}(T \times U)$$

denotes the set of all relations from T to U.

$$_ \leq _ \in \mathbb{N} \leftrightarrow \mathbb{N}$$

$$uses \ \in \{Jonathan, Mike, Peter\} \leftrightarrow Methods$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Domain and range

$$\begin{aligned}\mathrm{dom}\, R &= \{x : T \mid (\exists y : U \bullet (x,y) \in R)\} \\ \mathrm{ran}\, R &= \{y : U \mid (\exists x : T \bullet (x,y) \in R)\}\end{aligned}$$

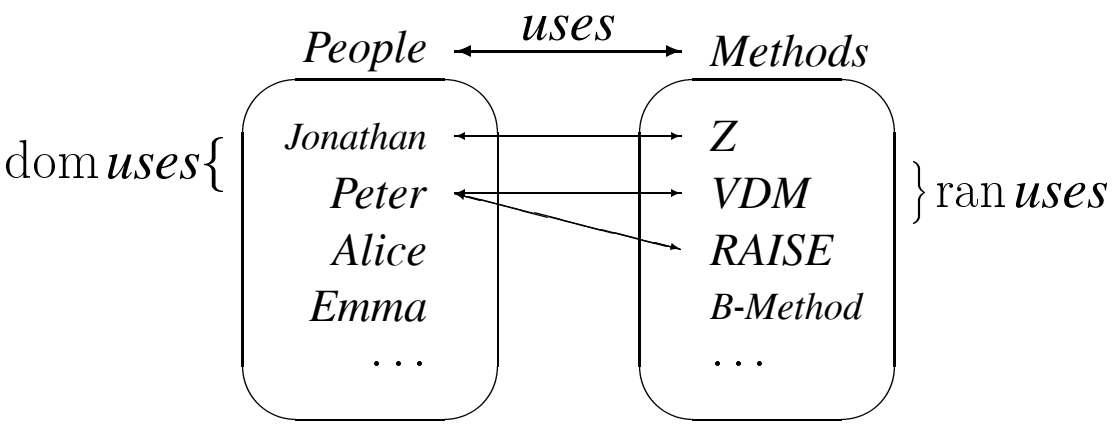
Inverse of a relation:

$$\begin{aligned}R^{\sim} &= \{x : T;\ y : U \mid (x,y) \in R \bullet (y,x)\} \\ &= \{x : T;\ y : U \mid (x,y) \in R \bullet y \mapsto x\} \\ &= \{y : U;\ x : T \mid (x,y) \in R\}\end{aligned}$$

If $R : T \leftrightarrow U$, then $R^{\sim}$ is of type $U \leftrightarrow T$.
$R^{\sim}$ may also be written as R^{-1}.

Examples:

$$\begin{aligned}\mathrm{dom}\, uses &= \{Jonathan, Peter\} \\ \mathrm{ran}\, uses &= \{Z, VDM, RAISE\} \\ uses^{\sim} &= \{Z \mapsto Jonathan, VDM \mapsto Peter, \\ &\quad RAISE \mapsto Peter\}\end{aligned}$$



Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Another example:

$$\begin{aligned}\operatorname{ran}_ \leq _ &= \operatorname{dom}_ \leq _ \\ &= \mathbb{N}\end{aligned}$$

$$(_ \leq _)^{\sim} = _ \geq _$$

Laws:

$$\operatorname{ran}(R^{\sim}) = \operatorname{dom} R$$

$$\operatorname{dom}(R^{\sim}) = \operatorname{ran} R$$

$$(R^{\sim})^{\sim} = R$$

⦚ N.B., relations are also sets.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Functions

Partial function:

$$X \pfun Y == \{f : X \rel Y \mid (\forall x : X; y_1, y_2 : Y \bullet (x \mapsto y_1) \in f \land (x \mapsto y_2) \in f \Rightarrow y_1 = y_2)\}$$

Total functions: '$\fun$' – domain covers all allowed values.

Function application

$f(x)$ applies the element x to the function f.

'$f\,x$' is the same as $f(x)$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Other standard functions in Z:

- $X \pfun\!\!\!\!\!\!\rightarrowtail Y$ – Partial injections (one-to-one mapping)
- $X \rightarrowtail Y$ – Total injections ($\mathrm{dom} f = X$)
- $X \twoheadrightarrow Y$ – Partial surjections ($\mathrm{ran} f = Y$)
- $X \rightarrow\!\!\!\!\!\!\twoheadrightarrow Y$ – Total surjections ($\mathrm{dom} f = X \land \mathrm{ran} f = Y$)
- $X \rightarrowtail\!\!\!\!\!\!\twoheadrightarrow Y$ – Bijections (one-to-one total injective and surjective, $f^{\sim} \in Y \rightarrow X$)
- $X \pfun\!\!\!\!\!\!\twoheadrightarrow Y$ – Finite partial functions (domain – and range – finite, subset of $X \pfun Y$ and $\mathbb{F}(X \times Y)$)
- $X \pfun\!\!\!\!\!\!\rightarrowtail Y$ – Finite partial injections (intersection of finite functions and partial injections)

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Toolkit operators

- $\mathrm{id}\, A$ – *Identity relation*, domain is mapped onto itself in the range, and vice versa.
- $Q \fatsemi R$ – *Forward relational composition* (or just 'relational composition'), range of relation Q composes with domain of relation R.
- $Q \circ R$ – *Backward relational composition*. Same as $R \fatsemi Q$.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

- $A \lhd R$ – *Domain restriction*. Set on the left hand side is used to restrict the relation's domain.
- $A \lhd\!\!\!\!- R$ – *Domain anti-restriction*. Complement of set is used for restriction.
- $R \rhd A$ – *Range restriction*. Range instead of domain is restricted by A.
- $R \rhd\!\!\!\!- A$ – *Range anti-restriction*. Hopefully you can work this out!
- $R(\!| A |\!)$ – *Relational image*. Domain of relation restricted like domain restriction.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

- $iter\, n\, R$ – *Relational iteration.* Relation composed with itself n times. (Often written R^n for brevity.) Domain and range must have same type (say A). Then:
  $iter\, 0\, R$ (R^0) is the same as $\mathrm{id}\, A$
  $iter\, 1\, R$ is the same as R
  $iter\, 2\, R$ is the same as $R \fatsemi R$
  $iter\, 3\, R$ is the same as $R \fatsemi R \fatsemi R$
  etc.
- $R^\sim$ – *Inverse of relation.* All tuples are reversed. Domain and range swapped. Same as R^{-1} if domain and range are type-compatible (i.e., relation is *homogeneous*).
- R^* – *Reflexive-transitive closure* (or just 'transitive closure'), union of all non-negative iterations of R – i.e., $R^0 \cup R^1 \cup R^2 \cup \ldots$
- R^+ – *Irreflexive-transitive closure.* Strictly positive iterations only. $R^+ = R^* \setminus R^0$.
- $Q \oplus R$ – *Overriding.* $(\mathrm{dom}\, R \lhd Q) \cup R$. Often used to modify function values.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Numbers

The set of integers:

$$\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$$

The set of natural numbers:

$$\mathbb{N} = \{n : \mathbb{Z} \mid n \geq 0\} = \{0, 1, 2, \ldots\}$$

Often used in Z specifications.

Arithmetic

addition	$_ + _$	$2 + 2 = 4$
subtraction	$_ - _$	$4 - 2 = 2$
multiplication	$_ * _$	$2 * 2 = 4$
division	$_ \,\mathsf{div}\, _$	$5 \,\mathsf{div}\, 2 = 2$
modulo arithmetic	$_ \,\mathsf{mod}\, _$	$5 \,\mathsf{mod}\, 2 = 1$
negation	$-$	$-(-2) = 2$

Brackets used for grouping; e.g., $2 * (4 + 5) = 18$.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Comparison relations

less than	$_ < _$	$2 < 3, \neg\ 3 < 2$
less than or equal	$_ \leq _$	$2 \leq 2$
greater than	$_ > _$	$a > b \Leftrightarrow b < a$
greater than or equal	$_ \geq _$	$a \geq b \Leftrightarrow b \leq a$

Maximum and minimum

maximum of a set $max\{1, 2, 3\} = 3$
minimum of a set $min\{1, 2, 3\} = 1$

≀≀≀ Note: the set must be non-empty and finite; otherwise the result is undefined.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Extra operators defined using an *axiomatic description*. E.g., absolute value of an integer:

$$
\begin{array}{|l}
abs : \mathbb{Z} \to \mathbb{Z} \\
\hline
\forall n : \mathbb{Z} \bullet \\
\quad n \leq 0 \Rightarrow abs\ n = -n \land \\
\quad n \geq 0 \Rightarrow abs\ n = n
\end{array}
$$

Note that $\mathrm{ran}\, abs = \mathbb{N}$.

Question: Can you suggest an alternative definition for *abs*? (Try using *max*.)

Axiomatic description available globally in the rest of a specification.

Description may be a *loose* specification (i.e., more than one possible model). E.g.:

$$
\begin{array}{|l}
n : \mathbb{N} \\
\hline
n \leq 10
\end{array}
$$

Any integer value of n from 0 to 10 allowable.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Strictly positive (non-zero) natural numbers:

$$\mathbb{N}_1 = \mathbb{N} \setminus \{0\} = \{1, 2, 3, \ldots\}$$

Exercise: Write definitions for:

1. the square of an integer,
2. the factorial of a natural number.

'Successor' function:

$$succ = \{0 \mapsto 1, 1 \mapsto 2, 2 \mapsto 3, \ldots\}$$

$\operatorname{ran} succ = \mathbb{N}_1$.

Inverse 'predecessor' function: $pred == succ^{\sim}$

Laws:

$$\begin{aligned}
succ &= \mathbb{N} \lhd (_ + 1) \\
&= (_ + 1) \rhd \mathbb{N}_1 \\
pred &= \mathbb{N}_1 \lhd (_ - 1) \\
&= (_ - 1) \rhd \mathbb{N} \\
succ \fatsemi succ &= \mathbb{N} \lhd (_ + 2) \\
&= (_ + 2) \rhd (\mathbb{N}_1 \setminus \{1\}) \\
succ \fatsemi pred &= \operatorname{id} \mathbb{N} \\
pred \fatsemi succ &= \operatorname{id} \mathbb{N}_1
\end{aligned}$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Iteration

$$R^n = R \fatsemi R \fatsemi \ldots \fatsemi R \quad k \text{ times}$$

Example, using $succ : \mathbb{N} \rightarrow \mathbb{N}$

$succ^0 = \mathrm{id}\, \mathbb{N}$

$succ^1 = succ$

$succ^2 = succ \fatsemi succ$

$succ^n = \mathbb{N} \lhd (_ + n) = (_ + n) \rhd \{i : \mathbb{N} \mid i \geq n\}$
for any $n : \mathbb{N}$

$succ^{-1} = succ^{\sim}$

Exercise: Provide another way of writing:

1. $R^m \fatsemi R^n$
2. $(R^m)^n$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Number range

For two integers $a, b : \mathbb{Z}$

$$a \mathinner{..} b = \{a, a+1, a+2, \ldots, b-2, b-1, b\}$$

More formally:

$$a \mathinner{..} b = \{n : \mathbb{Z} \mid a \leq n \leq b\}$$

If $a > b$ then $a \mathinner{..} b = \emptyset$.

Also $a \mathinner{..} a = \{a\}$.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Cardinality

Cardinality of a (finite) set is the number of elements.
Denoted $\#s$.

Thus $\#\emptyset = 0$, $\#\{a\} = 1$, $\#\{a, b\} = 2$ (if $a \neq b$), etc.

For number range $a \mathinner{..} b$,

$$\begin{aligned} \#a \mathinner{..} b &= 1 + b - a && \text{if } a \leq b \\ &= 0 && \text{if } a > b \\ &= max\{0, 1 + b - a\} \end{aligned}$$

For a finite set, $1 \mathinner{..} n$ can be mapped uniquely to each element;
n is the set's cardinality. E.g.:

$$\begin{array}{cccccc} 1 & 2 & 3 & \cdots & b-a & 1+b-a \\ \downarrow & \downarrow & \downarrow & \cdots & \downarrow & \downarrow \\ a & a+1 & a+2 & \cdots & b-1 & b \end{array}$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Types revisited

$\mathbb{Z}$ is a standard predefined basic type in Z.
$\mathbb{N} \subset \mathbb{Z}$ – not a true type.

$\mathbb{R}$ sometimes used for real numbers in Z. Often not needed.

Definition of a *basic type* (or *given set*):

$$[\, Places \,]$$

or for several sets,

$$[\, A, B, C \,]$$

A particular element:

$$London : Places$$

or for more than one element:

$$Oxford, Cambridge : Places$$
$$\hline$$
$$Oxford \neq Cambridge$$

⦚ Without the predicate, $Oxford = Cambridge$ would be possible!

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Free type definition:

$$Methods ::= Z \mid VDM \mid RAISE \mid B\text{-}Method$$

More complicated data type:

$$\mathbb{N} ::= zero \mid succ\langle\langle \mathbb{N} \rangle\rangle$$

Equivalent to:

$$[\mathbb{N}]$$

$$\begin{array}{|l} zero : \mathbb{N} \\ succ : \mathbb{N} \rightarrowtail \mathbb{N} \\ \hline \{zero\} \cap \operatorname{ran} succ = \emptyset \\ \{zero\} \cup \operatorname{ran} succ = \mathbb{N} \end{array}$$

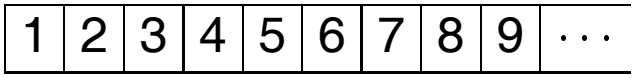
Functions like *succ* above known as *constructors*. Free type definitions not needed for many specifications.

Question:
What is the type of $\{i, j : \mathbb{N} \mid i < j\}$?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Sequences

E.g., lists, arrays, files, sequences, trace histories, . . . Elements indexed and contiguously numbered.



A sequence has 1st element, 2nd element, 3rd element, etc. . . (numbered from 1 rather than 0 in Z)

$$\mathrm{seq}\, T == \{s : \mathbb{N} \pfun T \mid \mathrm{dom}\, s = 1 \,..\, \#s\}$$

N.B.: sequences have a finite (but arbitrary) length.
Sequences are functions, which are relations, which are sets.

Length of sequence s is its cardinality, $\#s$.

Empty sequence, $s = \emptyset$ has $\#s = 0$.
Normally written as:

$\langle\rangle$ – the empty sequence

Like the empty set $\emptyset$, the empty sequence is typed.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Non-empty sequences:

$$\mathrm{seq}_1\, T == \mathrm{seq}\, T \setminus \{\langle\rangle\}$$

Injective sequences (i.e., no repeated elements):

$$\mathrm{iseq}\, T == \mathrm{seq}\, T \cap (\mathbb{N} \rightarrowtail T)$$

The sequence containing just one element, $s = \{1 \mapsto x\}$, has $\#s = 1$ and is written as

$\langle x \rangle$ – a singleton sequence

The sequence $\{1 \mapsto x_1, 2 \mapsto x_2, \ldots, n \mapsto x_n\}$ is normally written as

$\langle x_1, x_2, \ldots, x_n \rangle$ – a multi-element sequence

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Examples:

$\langle 11, 29, 3, 7 \rangle \in \mathrm{seq}\, \mathit{primes}$

$\langle \mathrm{J}, \mathrm{O}, \mathrm{N}, \mathrm{A}, \mathrm{T}, \mathrm{H}, \mathrm{A}, \mathrm{N} \rangle \in \mathrm{seq}\, \mathit{CHAR}$

Unlike for standard sets, two 'N' elements are distinct maplets $3 \mapsto$ N and $8 \mapsto$ N.

$\langle _\!\downarrow, \ulcorner, \leftrightarrows \rangle \in \mathrm{seq}\, \mathit{Path}$

Length (= cardinality) of above sequences is 4, 8 and 3 respectively.

⦚ N.B., unlike sets, sequences *can* have repeated elements. E.g.:

$$\langle \mathit{Emma} \rangle \neq \langle \mathit{Emma}, \mathit{Emma}, \mathit{Emma} \rangle$$

⦚ Also, the order is significant:

$$\langle \mathit{Alice}, \mathit{Emma} \rangle \neq \langle \mathit{Emma}, \mathit{Alice} \rangle$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Concatenation

For $s, t \in \operatorname{seq} T$

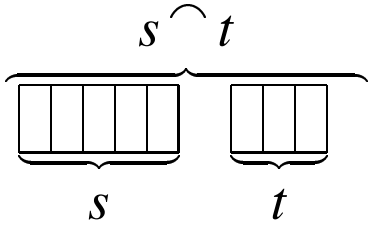
$s \frown t$ – the function $1 \mathinner{..} (\#s + \#t) \to T$
with elements

$$j \mapsto \begin{cases} s(j) & \text{if } 1 \leq j \leq \#s \\ t(j - \#s) & \text{if } \#s < j \leq (\#s + \#t) \end{cases}$$

More formally:

$$s \frown t = s \cup (_ - \#s) \fatsemi t$$

Concatenation of two sequences of length 5 and 3:



$\#s = 5 \quad \#t = 3$

$\#(s \frown t) = \#s + \#t = 5 + 3 = 8.$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Concatenation example:

$$\langle \mathsf{A} \rangle \frown \langle \mathsf{L}, \mathsf{I}, \mathsf{C}, \mathsf{E} \rangle =$$
$$\langle \mathsf{A}, \mathsf{L} \rangle \frown \langle \mathsf{I}, \mathsf{C}, \mathsf{E} \rangle =$$
$$\langle \mathsf{A}, \mathsf{L}, \mathsf{I}, \mathsf{C}, \mathsf{E} \rangle$$

$\langle a, b, c, \ldots \rangle$ is shorthand for $\langle a \rangle \frown \langle b \rangle \frown \langle c \rangle \frown \ldots$

Laws

$$\langle \rangle \frown s = s \frown \langle \rangle = s$$
$$r \frown (s \frown t) = (r \frown s) \frown t$$
$$(r \frown s = r \frown t) \Rightarrow s = t$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Prefix

For $s, t \in \operatorname{seq} T$, $s \subseteq t$ is equivalent to $\exists r : \operatorname{seq} T \bullet s \frown r = t$.

$\langle \mathsf{M}, \mathsf{A} \rangle \subseteq \langle \mathsf{M}, \mathsf{A}, \mathsf{N} \rangle$ (Freud's theorem!)
$\langle\rangle \subseteq s$ $\langle\rangle$ is always a prefix of any sequence.
$s \subseteq s$ A sequence is always a prefix of itself.

Laws

$$(s \subseteq t \land t \subseteq s) \Rightarrow s = t$$

$$(r \subseteq s \land s \subseteq t) \Rightarrow r \subseteq t$$

$$(r \subseteq t \land s \subseteq t) \Rightarrow (r \subseteq s \lor s \subseteq r)$$

These laws also apply to sets. A specific relation s prefix t is defined in the Z toolkit.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Other sequence operations

If $s \in \operatorname{seq} T$ and $s \neq \langle\rangle$ (i.e., $s \in \operatorname{seq}_1 T$):

First element:

$$head\ s = s(1) \qquad \bullet\circ\circ\circ\circ\circ$$

Last element:

$$last\ s = s(\#s) \qquad \circ\circ\circ\circ\circ\bullet$$

No first element:

$$tail\ s = succ\,^{\circ}_{9}(\{1\} \lhd s) \qquad \circ\bullet\bullet\bullet\bullet\bullet$$

No last element:

$$front\ s = \{\#s\} \lhd s \qquad \bullet\bullet\bullet\bullet\bullet\circ$$

Avoid applying these functions to an empty sequence; e.g., $head\langle\rangle$ is undefined. ≀≀≀

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

C_1	O_2	D_3	E_4

For $s = \langle \mathsf{C}, \mathsf{O}, \mathsf{D}, \mathsf{E} \rangle$
(which is $\{1 \mapsto \mathsf{C}, 2 \mapsto \mathsf{O}, 3 \mapsto \mathsf{D}, 4 \mapsto \mathsf{E}\}$)

$$head\ s = \mathsf{C}$$

$$last\ s = \mathsf{E}$$

$$\begin{aligned} tail\ s &= \{0 \mapsto 1, 1 \mapsto 2, 2 \mapsto 3, 3 \mapsto 4, \ldots\}^{\mathrm{g}}_{9} \\ &\quad \{2 \mapsto \mathsf{O}, 3 \mapsto \mathsf{D}, 4 \mapsto \mathsf{E}\} \\ &= \{1 \mapsto \mathsf{O}, 2 \mapsto \mathsf{D}, 3 \mapsto \mathsf{E}\} \\ &= \langle \mathsf{O}, \mathsf{D}, \mathsf{E} \rangle \end{aligned}$$

$$\begin{aligned} front\ s &= \{4\} \lhd \{1 \mapsto \mathsf{C}, 2 \mapsto \mathsf{O}, 3 \mapsto \mathsf{D}, 4 \mapsto \mathsf{E}\} \\ &= \{1 \mapsto \mathsf{C}, 2 \mapsto \mathsf{O}, 3 \mapsto \mathsf{D}\} \\ &= \langle \mathsf{C}, \mathsf{O}, \mathsf{D} \rangle \end{aligned}$$

Exercise: State some laws concerning these operations (and $_ \frown _$).

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

More general versions of *front* and *tail*, using *generic construction*:

$$
\begin{array}{|l}
\hline
[T] \\
\,\underline{for}\,, \\
\,\underline{after}\, : (\operatorname{seq} T) \times \mathbb{N} \to (\operatorname{seq} T) \\
\hline
\forall\, s : \operatorname{seq} T;\ n : \mathbb{N} \bullet \\
\quad s\,\underline{for}\,n = (1\,..\,n) \lhd s \ \land \\
\quad s\,\underline{after}\,n = (\{0\} \lhd succ^n) \fatsemi s \\
\hline
\end{array}
$$

E.g., for extracting portions of files as a sequence of bytes.

For $s \in \operatorname{seq}_1 T$,

$$front\ s = s\,\underline{for}\,(\#s - 1)$$

$$tail\ s = s\,\underline{after}\,1$$

Laws:

$$s\,\underline{for}\,0 = \langle\rangle$$

$$s\,\underline{for}\,\#s = s$$

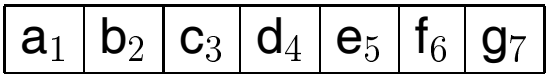
$$s\,\underline{after}\,0 = s$$

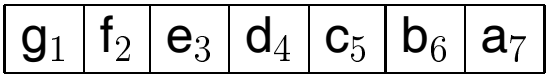
$$s\,\underline{after}\,\#s = \langle\rangle$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Reversal

Reverse of a sequence: $rev\ s$





Sequence $\langle \mathsf{D}, \mathsf{O}, \mathsf{G} \rangle$ is converted to $\langle \mathsf{G}, \mathsf{O}, \mathsf{D} \rangle$ using the *rev* function!

Laws

$$
\begin{aligned}
rev\langle\rangle &= \langle\rangle \\
rev\langle x \rangle &= \langle x \rangle \\
rev(rev\ s) &= s \\
rev(s \frown t) &= (rev\ t) \frown (rev\ s)
\end{aligned}
$$

For example

$$rev(\langle a, b \rangle \frown \langle c, d \rangle) = \langle d, c \rangle \frown \langle b, a \rangle$$

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Distributed operations

Concatenation of a sequence of sequences:

$$\frown/\langle\rangle = \langle\rangle$$

$$\frown/\langle a, b, \ldots, n\rangle = a \frown b \frown \ldots \frown n$$

More formally, $\frown/ : \mathrm{seq}(\mathrm{seq}\, T) \rightarrow \mathrm{seq}\, T$ satisfies

$$\frown/(\langle a\rangle \frown s) = \langle a\rangle \frown (\frown/\, s)$$

and also

$$\frown/(s \frown \langle a\rangle) = (\frown/\, s) \frown \langle a\rangle$$

Question: What is
$rev(\frown/\langle\langle \mathsf{N}, \mathsf{A}, \mathsf{H}\rangle, \langle \mathsf{T}, \mathsf{A}\rangle, \langle \mathsf{N}, \mathsf{O}\rangle, \langle \mathsf{J}\rangle\rangle)$?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Other distributed operators; e.g., overriding.

Informally:

$$\oplus/\langle a, b, \ldots, n\rangle = a \oplus b \oplus \ldots \oplus n$$

Exercise: Informally, the composition operator '$\fatsemi$' may also be generalized to a distributed composition operator '$\fatsemi/$':

$$\fatsemi/\langle a, b, \ldots, n\rangle = a \fatsemi b \fatsemi \ldots \fatsemi n$$

Write a formal definition for this. Can you think of an application?

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Disjointness and partitioning

Sequence of sets 'disjoint' if none of the sets in its range intersect:

$$\forall S : \operatorname{seq} \mathbb{P} T \bullet$$
$$\quad \text{disjoint } S \Leftrightarrow (\forall i, j : \operatorname{dom} S \mid i \neq j \bullet (S\,i) \cap (S\,j) = \emptyset)$$

Empty and singleton sequences of sets always disjoint (disjoint $\langle\rangle$ and disjoint $\langle x \rangle$ are always true).

For a two-element sequence, disjoint S is the same as:

$$S(1) \cap S(2) = \emptyset$$

For a three-element sequence, disjoint S is

$$S(1) \cap S(2) = \emptyset \land S(2) \cap S(3) = \emptyset \land S(3) \cap S(1) = \emptyset$$

etc.

Generalized union of all sets in the range of the sequence – if equal to a set, the sequence 'partitions' that set.

$$\forall S : \operatorname{seq} \mathbb{P} T;\ P : \mathbb{P} T \bullet$$
$$\quad S \text{ partition } P \Leftrightarrow (\text{ disjoint } S \land P = \bigcup \{ i : \operatorname{dom} S \bullet S\,i \})$$

Generalizable to any indexed set
(i.e., $S : I \pfun \mathbb{P} T$ instead of $S : \operatorname{seq} \mathbb{P} T$).

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Orders

A *partial order* is reflexive, antisymmetric and transitive:

$$
\begin{array}{l}
partial_order[X] == \\
\quad \{ _\underline{R}_ : X \leftrightarrow X \mid (\forall x, y, z : X \bullet \\
\qquad x \underline{R} x \land \\
\qquad (x \underline{R} y \land y \underline{R} x) \Rightarrow x = y \land \\
\qquad (x \underline{R} y \land y \underline{R} z) \Rightarrow x \underline{R} z) \}
\end{array}
$$

A *total order* relates all elements:

$$
\begin{array}{l}
total_order[X] == \\
\quad \{ _\underline{R}_ : partial_order[X] \mid (\forall x, y : X \bullet \\
\qquad x \underline{R} y \lor y \underline{R} x) \}
\end{array}
$$

E.g., time could be modelled as a total order.

Copyright © 1996,1998 Jonathan Bowen. All rights reserved.

Summary

Sequences: an important data structure in many specifications and data refinement.

Some lesser used parts of Z (such as *bags*) have not been covered.

Mathematics for specification – also need the *schema* notation for structuring and modularization of specifications.